

Statistics 8330 Final Exam – Fall 2022

In order to full credit, your code must include comments that explain what you are doing. You may use all sources of information except direct communication with other people. There will be partial credit for successfully doing portions of problems, even if you don't do everything. Upload your completed exam to the Final Exam drop box on eLC. Each problem is worth equal points. You may use R or Python for all problems. Good luck and happy holidays!

1. In this problem, you will write a function to find the distribution of order statistics by simulation. The k th order statistic for a data set is the k th data point when ordered from lowest to highest. The call for the function should be `ordersim(x,k,n,mu1,sd1,nrep,lower.tail)`. It should output the probability that the k th order statistic is less than or equal to x for n data points from a normal distribution with mean μ_1 and standard deviation sd_1 . The probability should be calculated using simulations with $nrep$ replicates. If `lower.tail` equals true, then it should calculate $P(X \leq x)$. If `lower.tail=F`, then it should calculate $P(X > x)$. It should have a default of `lower.tail=T`.
2. We discussed loess regression in class and you also wrote a function in homework #5 to do loess regression. Recall that loess regression requires a smoothing parameter that determines how it weights nearby data points versus more distant data points in fitting the value at each point. In this problem, you will write a function that finds the value of the smoothing parameter by cross-validation. Your function should be of form `findspan(dat,smoothvec,k,plotY)`, where `dat` is a data frame with the single predictor in column 1 and the response variable in column 2, `smoothvec` is the vector of values for the smoothing parameter that the function will search, `k` specifies the number of folds in k-fold cross validation, and `plotY` is a TRUE/FALSE variable determining whether the function makes a plot. The output should be the optimal value of the smoothing parameter. Also, if `plotY=TRUE`, then it should produce a plot of the data with the optimal loess (as determined by your function) curve plotted as well. You should use the `loess` function to do loess regression. The `span` parameter in `loess` is the smoothing parameter. Demonstrate your function on simulated data.
3. In class we discussed the maximal margin classifier for the case of two classes and separable data. As we discussed, this method finds the hyperplane separating the two classes that has the largest margin. In this problem, you will write a function that implements a version of this method. In its usual form, the maximal margin classifier requires solving the optimization problem of finding the hyperplane with the largest margin. We have not discussed optimization in this class (see STAT 8060!) and so we will NOT do this. Instead, we will approach this using randomization. You can limit to the two-dimensional case (i.e. two independent variables plus the classification variable). The basic approach is this:

(CONTINUE TO THE NEXT PAGE)

Loop over lines

```
{ 1. generate a random hyperplane (i.e. a line)
  2. Find out if all points are classified correctly. If not, return to step 1
  3. If all points are classified correctly, calculate the distance from the
     data points to the hyperplane.
  4. Find the minimum distance from step 3. This is the margin.
}
```

The hyperplane that produces the largest margin is the best one.

The function should be of form **MaxMargClass(x,y,nlines=100,bmean=1,bsd=3,makePlot=T)** where **x** is the data matrix, **y** is the response vector, **nlines** is the number of random lines to generate. The arguments **bmean** and **bsd** specify the mean and standard deviation of the random slopes. You may generate your random lines differently and so have different arguments for these.

The output should be a labeled list that contains the coefficients of the maximal margin classifier and the value of the margin. If **Makeplot** is **TRUE** then it should make a plot showing the data and the classifier. Demonstrate your function with sample data that you simulate. Here is an example of running it:

```
MaxMargClass(x3,y3,nlines=1000)
```

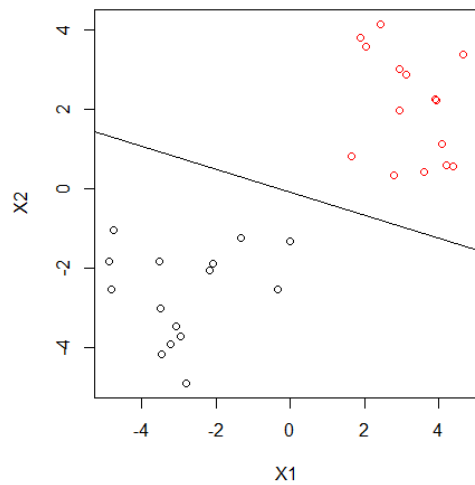
```
$`best line`
```

```
[1] -0.2668654 0.3311341 0.2336626
```

```
$M
```

```
[1] 1.996554
```

(CONTINUE TO THE NEXT PAGE)



Notes:

- 1) This is a very inefficient way to find the solution. It works reasonably well in two dimensions but would not work at all in higher dimensions.
- 2) The distance from a point (X_{i1}, X_{i2}) to the hyperplane defined by coefficients $(\beta_0, \beta_1, \beta_2)$ is given by

$$d = \frac{|\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2}|}{(\beta_0^2 + \beta_1^2 + \beta_2^2)}$$
- 3) You can decide how to generate the random lines. Anything reasonable is fine.